

Storm that Castle:

Fight to be King of the HILL

Final Report

TEAM 12

TREBUCHET | STORM

TEAM MEMBERS

Jagannathan Ranganathan, Sukesh
Vazhunnavar, Manu Govind
Cho, Jaemin
Ghoshal, Pritam
Kale, Ashish

INSTRUCTOR

Prof. Laura Blumenschein

SUBMISSION DATE

May 4, 2026

INSTITUTION

Purdue University
School of Mechanical Engineering

ABOUT THE TREBUCHET

An autonomous robot designed for the STORM (Send Teams Orbs to Remain Monarchs) competition. The bot navigates a 4 ft x 8 ft field using IR line-following, detects HILL allegiance via IR beacon frequency, and deploys 1.65-inch foam UNITs using a brushless-motor launcher with a servo-controlled hopper gate. Motion is governed by a Finite State Machine running on 2 Arduino Uno MCUs, each with a PCA9685 servo driver.

ABSTRACT

The report presents the design and development of an autonomous robot for the ME 588 STORM competition, in which two robots compete to control HILLs by launching foam balls into opponent-controlled buckets. Our robot uses a dual-Arduino architecture: one Arduino handles the user interface, game timer, team selection, and IR beacon sensing, while the second controls line following, junction detection, and ball launching. The robot navigates a taped path using a SparkFun 8-sensor IR array with PID control. At the start of each run, a 3-second grace period allows the ESC to arm, followed by a brief forward drive to bypass the home base junction. The robot then follows the line and stops at each of the four junctions where perpendicular lines point toward HILLs. At each stop, it waits 5 seconds while Arduino 1 continuously broadcasts IR beacon classifications (750 Hz for blue, 1500 Hz for red). If opponent votes outnumber self votes and at least three opponent signals were detected, a BLDC flywheel launcher spins up, and a servo-driven flap releases a ball. Otherwise, the junction is skipped. After visiting all four junctions, the robot stops and waits for the next game to start. Two daisy-chained PCA9685 servo drivers run at different frequencies to independently control the drive motors and the launcher.

INTRODUCTION

Overview of Project Objective

The STORM (Send Teams Orbs to Remain Monarchs) competition requires each team to build a fully autonomous robot that navigates a 4 ft by 8 ft playing field, identifies opponent-controlled HILLs, and delivers foam balls into them to flip their allegiance. HILLs are square buckets positioned along the side rail of the field, 12 inches above the playing surface, and each one broadcasts its current allegiance using an IR LED beacon at a set frequency: 750 Hz for blue and 1500 Hz for red. A match lasts 2 minutes and 30 seconds, and the team controlling the most HILLs when time expires wins. The robot must start within a 12-inch cube at the HOME BASE, begin operation with a single button press, and stop all movement autonomously when the timer runs out. Black electrical tape lines on the field connect the HOME BASE to perpendicular branches that point at each HILL location. The robot is expected to follow these lines, recognize junctions, and make launch decisions without any human input during the match.

Brief Introduction of Solution

Our approach splits the problem into two cooperating subsystems running on separate Arduinos connected over serial. Arduino 1 serves as the operator interface and sensor hub: it reads two toggle switches (system on/off and team color), runs the 2:30 game timer on an LCD, and continuously monitors IR beacon frequencies using interrupt-driven pulse counting on dedicated left and right sensors. Every 250 milliseconds, it measures the active sensor's pulse count, classifies the frequency as self, opponent, or no signal, and broadcasts the result to Arduino 2 every 500 milliseconds. Arduino 2 handles all motion and launches autonomously. After a 3-second startup grace period for ESC arming, it drives forward briefly to clear the home base junction line, then begins PID line following. When it detects a junction (five or more sensors active simultaneously), it stops and waits up to 5 seconds, collecting the continuous stream of target classifications from Arduino 1. It counts the number of opponent and self-votes it received, ignoring no-signal readings. If opponent votes outnumber self-votes and at least three opponent signals were detected, it launches a ball. Otherwise, it skips the junction and continues. After visiting all four junctions, the robot stops permanently and waits for the next START command. This split architecture keeps timing-sensitive IR frequency measurement isolated from the I2C and PWM control on the other Arduino.

Brief Introduction of Robot Design, Sensors/Actuators, Control/Motion Strategy, and Calibration

The robot is built on a custom chassis that houses two DC geared motors, driven by an L298N H-bridge. Speed and direction signals are generated by a PCA9685 servo driver at 0x40 running at 1000 Hz, giving smooth PWM control without audible motor whine. A second PCA9685 at

0x41 runs at 50 Hz and handles the launcher subsystem: a bidirectional BLDC motor controlled through an ESC (throttle and direction channels) and a micro servo that actuates a flap to release balls toward the correct side. Line following uses a SparkFun 8-sensor IR reflectance array at I2C address 0x3E, with a weighted-average position calculation feeding a PD controller ($K_P = 0.35$, $K_D = 0.4$). Junction detection triggers when five or more sensors are active simultaneously, with an 800 ms cooldown to prevent double-counting. The ESC requires a specific arming sequence at startup: 3 seconds of zero throttle at a fixed reference angle of 90 degrees, after which the team-specific direction angle (0 degrees for blue, 180 degrees for red) is applied. A brief forward drive immediately after arming clears the home base junction line, so it is not mistakenly counted as the first HILL. Calibration involved tuning PID gains for stable line following, setting junction sensor thresholds to reliably detect perpendicular tape crossings, adjusting launcher throttle and flap timing for consistent ball release, and determining the correct voting thresholds to avoid false launches from transient IR noise.

BODY

Detailed Robot Design

The robot is built around a dual-layer chassis structure made of pre-fabricated steel plates. The lower level consists of two parallel steel chassis plates mounted vertically with a 3D-printed battery holder bracket positioned between them. This vertical orientation creates a central cavity for the 12V battery pack while maintaining structural rigidity through the steel frame. Four DC geared motors with yellow rubber wheels are mounted at the corners of the chassis, providing four-wheel drive. The motors are connected to an L298N H-bridge driver that receives PWM speed signals from the PCA9685 board #1, running at 1000 Hz. A SparkFun 8-sensor IR line follower array is mounted at the front edge of the chassis, positioned low enough to detect the black electrical tape on the white playing surface.

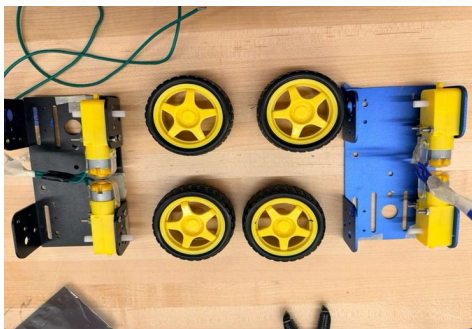


Figure 1: Dual metal chassis system with motors

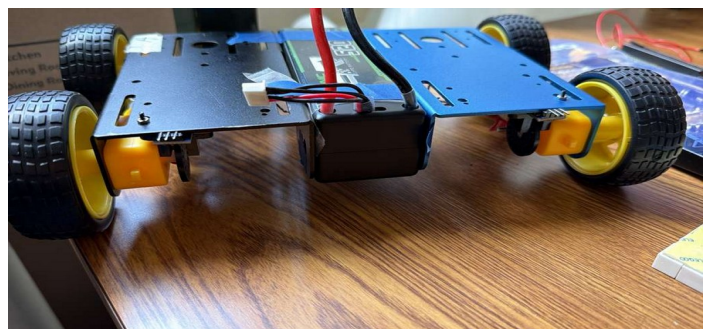


Figure 2: Assembled chassis with battery

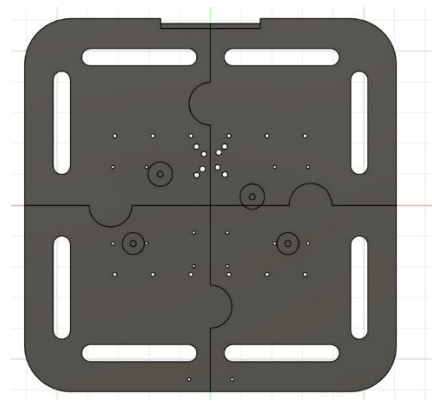


Figure 3: 3D printed chassis base



Figure 4: Battery holder

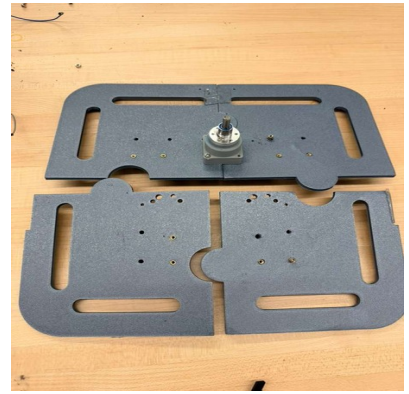


Figure 5: BLDC mounted on base

Above the battery cavity, a 3D-printed horizontal base plate provides the mounting surface for the launcher assembly. A BLDC drone motor is rigidly attached to this base with its shaft oriented vertically. A 24-tooth spur gear (3D-printed from the drive-gear STEP file) is mounted on the motor shaft, serving as the driver gear in the launcher mechanism. A second 24-tooth spur gear (the driven gear STEP file), coupled to a flywheel base and wrapped with a TPU sleeve for improved ball grip, acts as the driven gear and is positioned adjacent to the motor gear. The TPU sleeve increases friction between the flywheel surface and the foam ball during launch. The gear mesh drives the flywheel up to launch speed when the ESC commands the motor to spin. Between the two gears, a 3D-printed curved ramp is angled so that balls rolling down from the hopper pass through the gear mesh and are accelerated by contact with the rotating TPU-wrapped flywheel surface. The ramp's curvature directs the ball's trajectory left or right, depending on the motor's rotation direction, which is controlled by the ESC's direction servo channel.

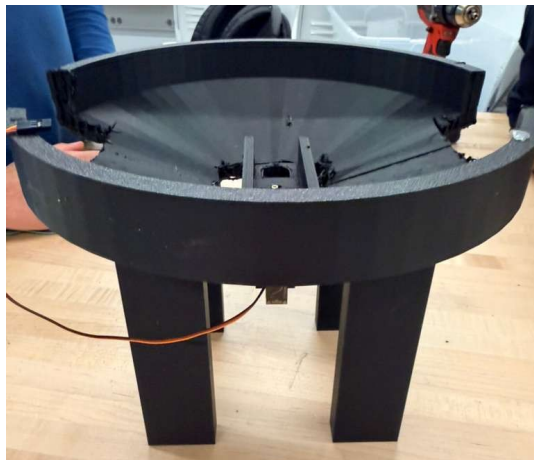


Figure 6: 3D printed hopper



Figure 7: Ball flap

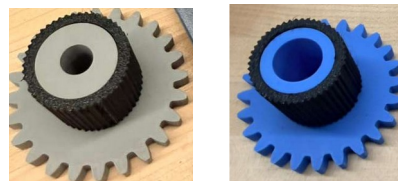


Figure 8: Driven and drive gear



Figure 9: Launch ramp

At the top of the assembly, a large conical hopper made from two 3D-printed parts (Hopper_part_1 and Hopper_part_2, printed in black PLA and joined together) houses the foam balls before launch. The hopper walls are angled inward to funnel balls toward a central opening directly above the flywheel and ramp. At the base of the hopper, a servo-actuated flap

(ball_holder_flap.STL, printed in blue material) controls ball release. The flap is shaped like a wedge or gate. When the servo rotates to one extreme angle (15 degrees for the blue team, 165 degrees for the red team), the flap pivots aside, allowing a ball to drop through the opening onto the ramp. When the servo returns to the center position (90 degrees), the flap blocks the opening, preventing additional balls from dropping. This gated release ensures that only one ball is launched per firing sequence, and the flap angle determines which side of the ramp the ball lands on, so it exits toward the correct HILL bucket.

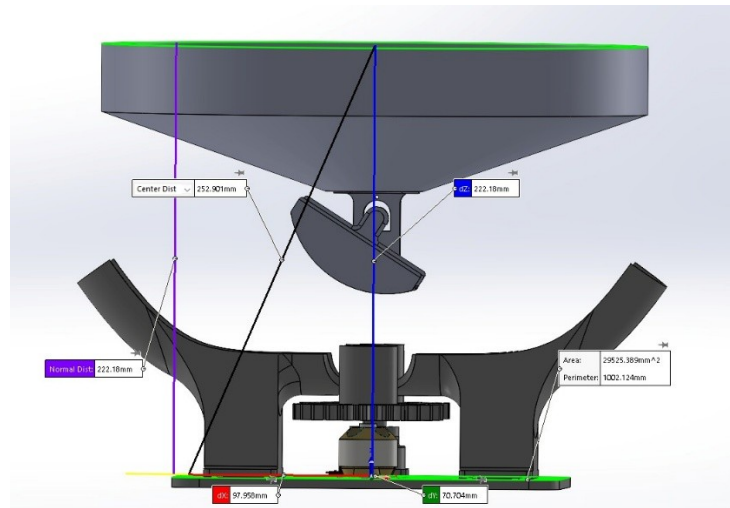


Figure 10: Entire assembly

The electronics are distributed across two levels. Arduino 1, the LCD module with its 3D-printed mounting bracket (Body1_wdisplay and Body2_wdisplay), toggle switches, and the two IR flame sensor modules are mounted on or near the top deck for easy operator access. Arduino 2, both PCA9685 boards, and the L298N driver are mounted lower on the steel chassis to minimize the distance to the drive motors. An LM2596 buck converter steps the 12V battery voltage down to 5V for the logic and sensor power rails. All I2C devices share a common bus: the line sensor array at 0x3E, PCA #1 at 0x40, PCA #2 at 0x41, and the LCD's I2C backpack at 0x27. Power distribution uses a star topology with heavy-gauge wires running from the battery to the buck converter, the L298N motor supply input, and the ESC power input, with thinner wires branching to the 5V logic devices. The entire assembly fits within a 12-inch cube envelope when measured from the bottom of the wheels to the top of the hopper.

Detailed Motion Planning

The robot navigates the playing field using a weighted-average line-following algorithm with the SparkFun 8-sensor IR reflectance array. Each sensor is assigned a position coordinate from -3.5 to +3.5 relative to the array center. The software sums the positions of all active sensors and divides by the count to produce a signed error value. This error feeds a PD controller ($K_P = 0.35$, $K_D = 0.4$) that generates a correction term added to the left motor and subtracted from the right motor, steering the robot back toward the line. When centered, both motors run at the base speed of 1100 PWM counts. Junction detection triggers when five or more sensors activate simultaneously. The robot then drives forward for 200 milliseconds to position the launcher over

the junction, stops, and begins the target-voting phase. After launching or skipping, it resumes with a brief speed boost. At startup, a 750-millisecond forward drive bypasses the home base junction line, with detection disabled for the first 800 milliseconds to prevent miscounting.

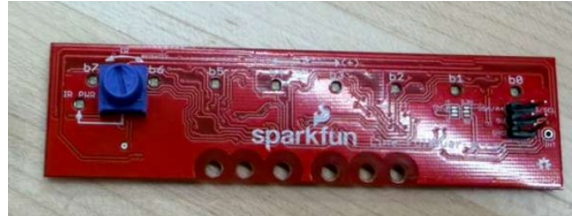


Figure 11: SparkFun IR Sensor Array

Detailed Sensor and Actuator Calibration

The IR beacon detection system required careful frequency discrimination to distinguish between the 750 Hz blue hill beacons and 1500 Hz red hill beacons. Two flame sensor modules were mounted on opposite sides of the robot, with the right sensor monitoring red hills and the left sensor monitoring blue hills. Each sensor's output triggers an interrupt on Arduino 1 that increments a pulse counter. Every 250 milliseconds, the software calculates the frequency and classifies it using empirically tuned windows of 600-900 Hz for the 750 Hz beacon and 1300-1700 Hz for the 1500 Hz beacon. The operator interface consists of three switches and a 16x2 multicolor LCD. A main toggle switch on pin D4 controls start and stop, a second toggle on pin D5 selects team color (latched at startup to prevent mid-game changes), and a hardware emergency stop switch sits in series with the power supply.

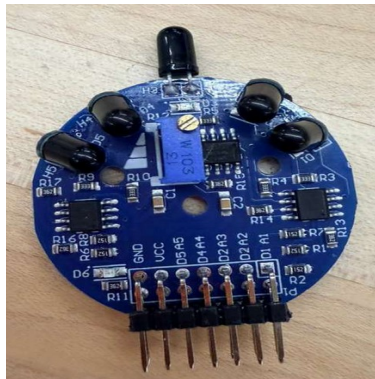


Figure 12: Flame Sensor



Figure 13: Emergency Switch



Figure 14: 2 Toggle switches

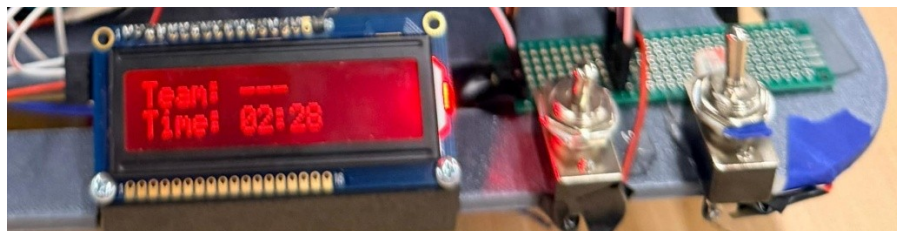


Figure 15: Display

The LCD connects via I2C at address 0x27 and displays IR detection state and countdown timer, with backlight color matching the selected team. Pin D8 controls LCD color using ME 588 TEAM 12

INPUT_PULLUP for red and OUTPUT LOW for blue, providing stable switching. The SparkFun 8-sensor line follower array was calibrated by testing different thresholds, settling on five active sensors for reliable junction detection with an 800-millisecond cooldown to prevent double-counting. Each sensor was assigned a position coordinate from -3.5 to +3.5 for weighted-average error calculation.

PID controller gains were tuned iteratively on the field, with KP increased to 0.35 for responsive steering and KD set to 0.4 to dampen oscillations during curve-to-straight transitions. Integral gain remained at zero since sustained drift was never observed. Base speed was set to 1100 PWM counts, curve speed to 1500 when line position exceeded 1.0 units from center, and pivot speed to 3000 for sharp turns above 1.8 units. A 0.05-unit center deadband prevents unnecessary corrections. The ESC arming sequence required precise timing: during the 3-second arming window, the direction signal is held at 90 degrees neutral while throttle remains at zero, and only after arming completes does the software apply the team-specific direction angle (0 degrees for blue, 180 degrees for red). Attempting to arm the team angle directly caused incorrect ESC latching. Launch throttle was calibrated incrementally from 0.20 to 0.30, settling at 0.27 to ensure consistent delivery into 12-inch-high HILL buckets. Flap servo angles (15 degrees for blue, 165 degrees for red) were determined by manual rotation until the flap fully cleared the hopper opening. The 600-millisecond flap hold duration allows complete ball release, with a second 600-millisecond hold at center, ensuring the servo physically returns before the next cycle.

Detailed Control Strategy

The robot's control system is implemented as a finite-state machine with 9 states (S0-S8) controlled by an 8-bit input register. Each input bit represents a specific condition: bit 0 (START_CMD) signals a start request from the operator, bit 1 (STOP_CMD) signals a stop request, bit 2 (JCT_DETECT) is high when five or more line sensors are active, bit 3 (T_ARM_DONE) is set when the 3-second ESC arming timer expires, bit 4 (T_GRACE_DONE) is set after the 10-second startup grace period, bit 5 (T_BYPASS_DONE) is set after the 750-millisecond home base bypass drive, bit 6 (T_JCT_DONE) is set after the 200-millisecond junction alignment drive, and bit 7 (VOTE_FIRE) is set when the IR voting condition ($OPP_VOTES > SELF_VOTES$ AND $OPP_VOTES \geq 3$) evaluates true. A separate 4-bit junction counter tracks the number of junctions processed. State outputs control the drive motors (O0), the ESC direction servo (O1), the ESC throttle (O2), the flap servo (O3), the PID controller activation (O4), the vote collection logic (O5), and the junction counter increment (O6).

State transitions are triggered by specific input bit patterns shown in hex. From S0 (IDLE), the START signal alone (input = 0x01) advances to S1 (ESC_ARMING), where the ESC direction is held at 90 degrees with zero throttle. The arming timer expiration (0x08) moves to S2 (GRACE), the grace timer (0x10) advances to S3 (HOME_BYPASS), and the bypass timer (0x20) reaches S4 (LINE_FOLLOW). In S4, junction detection (0x04) transitions to S5 (JCT_DRIVE), which advances the alignment timer (0x40) to S6 (IR_VOTE). From S6, a successful vote (0x80) launches the ball through S7, while a failed vote or timeout (0x00) skips

back to S4. After four junctions (JCT_CNT=4), S4 transitions to the terminal state S8 (STOPPED). A global override applies to every state: STOP_CMD (0x02) immediately returns the FSM to S0, killing all actuators regardless of current operation. This non-blocking state machine runs on Arduino 2 and evaluates all input bits each loop iteration, ensuring the STOP override is honored within milliseconds of receipt.

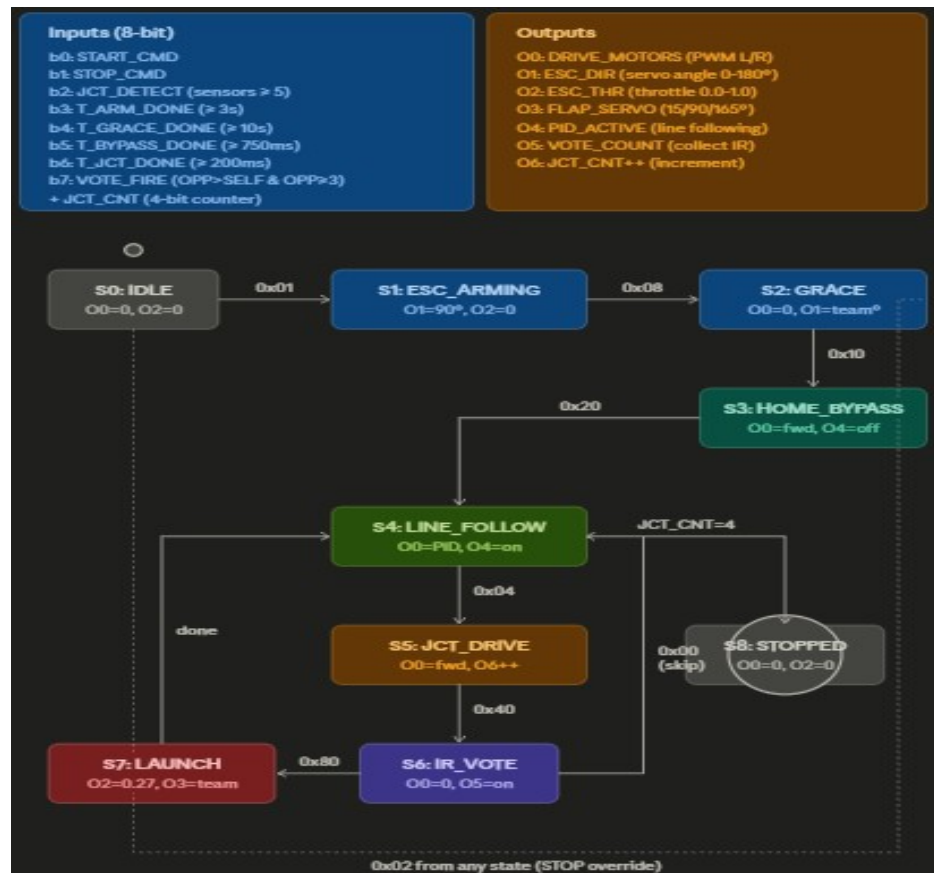


Figure 16: High-level FSM

Results

The launcher ramp required the most iterations of any single component. Its curved geometry had to send the foam ball into the HILL bucket, 12 inches above the playing surface, and early prints either launched too low (the ball striking the bucket wall) or too high (overshooting entirely). Several adjustments to the ramp curvature and exit angle, along with corresponding 3D-print iterations, eventually produced a profile that consistently delivered balls into the bucket from the junction-stopping positions. The servo-controlled flap also went through a redesign. The first version was too thick and protruded into the ball's exit path, causing balls to catch on its edge and jam during launch. A thinner, redesigned flap cleared the trajectory completely once the servo reached the team angle, after which ball release became reliable across both team configurations.

Sensor calibration required tuning the onboard potentiometers on the SparkFun line array and both IR flame sensor modules. The line array threshold was adjusted to reliably distinguish black tape from the white floor under testing-area lighting. At the same time, the flame sensors were tuned to detect the beacon LEDs at a distance in the field without picking up ambient IR noise. Both adjustments were made by observing live serial output with known inputs. Drive speed was the final calibration step: speeds below 900 PWM counts caused stalling on starts and during curves, while speeds above 1500 PWM counts caused oscillation and missed junctions. The final base speed of 1100 PWM counts gave smooth tracking with reliable junction detection. Across multiple full-match runs, the robot reliably executed the start sequence, navigated to all four HILL junctions, made correct launch decisions from IR classifications, and delivered balls into opponent-controlled buckets within the 2:30 game timer.

Discussion

The completed robot met all major project objectives: it operated autonomously after a single button press, followed the taped path using PID control, identified opponent-controlled HILLS via IR beacon frequency classification, and accurately launched foam balls into target buckets, all within the 2:30 match window. Two significant constraints shaped the final design. The first was a signal conflict on the PCA9685 servo driver: the drive motors required high-frequency PWM (1000 Hz) to avoid audible whine and provide smooth control, while the BLDC ESC and flap servo required standard 50 Hz servo pulses. Initial attempts to switch a single PCA9685 between frequencies on demand caused glitches in the ESC direction signal, preventing bidirectional mode from engaging reliably. We resolved this by adopting a dual PCA9685 architecture, with one board permanently set to 1000 Hz for the L298N drive motors and a second board fixed at 50 Hz for the launcher and flap. The second constraint involved the bidirectional ESC's arming behavior: the ESC latched its direction reference from the yellow signal wire during the arming window, so arming with the team-specific direction angle (0° or 180°) caused the ESC to lock out the opposite direction, making it impossible to switch teams without rebooting. The fix was to always arm at a fixed neutral reference angle of 90°, regardless of team, then switch to the team-specific direction only after the 3-second arming sequence completed. This made team color switching symmetric and reliable, and removed a class of unpredictable launch failures that had plagued earlier integration tests.

Conclusion

The team successfully designed, built, and tested an autonomous robot that meets all functional requirements of the STORM competition: line following, junction detection, IR beacon classification, ball launching, and start/stop control within the 2:30 match window. The dual-Arduino architecture proved effective for separating timing-sensitive sensing from actuator control, and the dual-PCA9685 setup resolved the frequency conflict between drive motors and the launcher subsystem. Future work would focus on adding a reverse-patrol mode to revisit junctions during the match, improving launch consistency with a closed-loop ESC throttle controller, and reducing the 5-second IR voting window by enabling faster sensor sampling for more responsive launch decisions.

Appendix

Our team successfully cleared all requirements for the project checkoff on 29th April 2026.

Arduino_1.ino

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <SoftwareSerial.h>

// =====
// SERIAL TO ARDUINO 2
// =====
//
// Arduino 1 D10 = RX, connect to Arduino 2 D11
// Arduino 1 D11 = TX, connect to Arduino 2 D10
//
const byte ARDUINO2_RX_PIN = 10;
const byte ARDUINO2_TX_PIN = 11;

SoftwareSerial arduino2Link(ARDUINO2_RX_PIN, ARDUINO2_TX_PIN);

// =====
// LCD
// =====
// Change 0x27 to 0x3F if needed.
LiquidCrystal_I2C lcd(0x27, 16, 2);

// =====
// PINS ON ARDUINO 1
// =====

const byte SENSOR1_PIN = 2;          // Blue team active sensor
const byte SENSOR2_PIN = 3;          // Red team active sensor

const byte START_SWITCH_PIN = 4;     // Start/Stop toggle
const byte TEAM_SWITCH_PIN = 5;      // Team color toggle

const byte LCD_BLUE_CTRL_PIN = 8;    // LCD pin 18 blue control

// =====
// SETTINGS
// =====

const unsigned long GAME_TIME_MS     = 150000UL; // 2 min 30 sec
const unsigned long MEASURE_WINDOW_MS = 250UL;
const unsigned long LCD_UPDATE_MS     = 200UL;
const unsigned long COMMAND_REPEAT_MS = 50UL;
const unsigned long STATUS_SEND_MS    = 500UL;

const float FREQ_750_MIN = 600.0;
const float FREQ_750_MAX = 900.0;
const float FREQ_1500_MIN = 1300.0;
const float FREQ_1500_MAX = 1700.0;

// =====
// STATE
// =====

volatile unsigned long pulseCount1 = 0;
volatile unsigned long pulseCount2 = 0;

unsigned long gameStartTime = 0;
unsigned long lastMeasureTime = 0;
unsigned long lastLcdUpdate = 0;
unsigned long lastStatusSend = 0;
```

```

bool systemIsOn = false;
bool gameOver = false;
bool lastStartSwitchState = false;

enum TeamColor {
    BLUE_TEAM = 0,
    RED_TEAM = 1
};

enum DetectionState {
    NO_SIGNAL = 0,
    DETECT_SELF,
    DETECT_OPPONENT
};

TeamColor currentTeam = BLUE_TEAM;
TeamColor lastSentTeam = BLUE_TEAM;

DetectionState currentDetection = NO_SIGNAL;
DetectionState lastSentDetection = NO_SIGNAL;

// Non-blocking repeated command sender
bool repeatCommandActive = false;
const char* repeatCommandText = "";
unsigned int repeatCommandRemaining = 0;
unsigned long lastRepeatCommandTime = 0;

// =====
// INTERRUPTS
// =====

void countPulse1() {
    pulseCount1++;
}

void countPulse2() {
    pulseCount2++;
}

// =====
// SWITCH HELPERS
// =====
//
// INPUT_PULLUP logic:
// Switch connected to GND = ON / LOW
//

bool startSwitchOn() {
    return digitalRead(START_SWITCH_PIN) == LOW;
}

TeamColor getTeam() {
    if (digitalRead(Team_SWITCH_PIN) == LOW) {
        return BLUE_TEAM;
    } else {
        return RED_TEAM;
    }
}

// =====
// LCD COLOR
// =====
//
// LCD color behavior:
// default = RED
// grounding LCD pin 18 = BLUE
//

void setLCDColor(TeamColor team) {
    if (team == BLUE_TEAM) {
        pinMode(LCD_BLUE_CTRL_PIN, OUTPUT);
        digitalWrite(LCD_BLUE_CTRL_PIN, LOW);
    }
}

```

```

    } else {
        pinMode(LCD_BLUE_CTRL_PIN, INPUT);
    }
}

// =====
// LCD HELPERS
// =====

void lcdSystemOff() {
    lcd.noBacklight();
    lcd.clear();
}

void lcdSystemOn() {
    lcd.backlight();
    lcd.clear();
}

void printLine(byte row, String text) {
    const int LCD_WIDTH = 16;

    if (text.length() < LCD_WIDTH) {
        while (text.length() < LCD_WIDTH) {
            text += " ";
        }
    } else if (text.length() > LCD_WIDTH) {
        text = text.substring(0, LCD_WIDTH);
    }

    lcd.setCursor(0, row);
    lcd.print(text);
}

String formatTimeRemaining(unsigned long elapsedMs) {
    unsigned long remainingMs = 0;

    if (elapsedMs < GAME_TIME_MS) {
        remainingMs = GAME_TIME_MS - elapsedMs;
    }

    unsigned long totalSeconds = remainingMs / 1000;
    int minutes = totalSeconds / 60;
    int seconds = totalSeconds % 60;

    char buf[17];
    sprintf(buf, "Time: %02d:%02d", minutes, seconds);
    return String(buf);
}

String detectionText(DetectionState state) {
    if (state == DETECT_SELF) {
        return "Team: Self";
    }

    if (state == DETECT_OPPONENT) {
        return "Team: Opponent";
    }

    return "Team: ---";
}

// =====
// SENSOR HELPERS
// =====

void resetPulseCounters() {
    noInterrupts();
    pulseCount1 = 0;
    pulseCount2 = 0;
    interrupts();
}

```

```

float getActiveSensorFrequency(TeamColor team) {
    unsigned long count1;
    unsigned long count2;

    noInterrupts();
    count1 = pulseCount1;
    count2 = pulseCount2;
    pulseCount1 = 0;
    pulseCount2 = 0;
    interrupts();

    unsigned long activeCount = 0;

    if (team == BLUE_TEAM) {
        activeCount = count1;
    } else {
        activeCount = count2;
    }

    return (activeCount * 1000.0) / MEASURE_WINDOW_MS;
}

DetectionState classifyFrequency(float freq, TeamColor team) {
    bool is750 = (freq > FREQ_750_MIN && freq < FREQ_750_MAX);
    bool is1500 = (freq > FREQ_1500_MIN && freq < FREQ_1500_MAX);

    if (!is750 && !is1500) {
        return NO_SIGNAL;
    }

    if (team == RED_TEAM) {
        // Bucket loyal to RED blinks at 1500 Hz.
        // So for RED team:
        // 1500 Hz = Self
        // 750 Hz = Opponent
        if (is1500) return DETECT_SELF;
        if (is750) return DETECT_OPPONENT;
    } else {
        // Bucket loyal to BLUE blinks at 750 Hz.
        // So for BLUE team:
        // 750 Hz = Self
        // 1500 Hz = Opponent
        if (is750) return DETECT_SELF;
        if (is1500) return DETECT_OPPONENT;
    }

    return NO_SIGNAL;
}

// =====
// SERIAL COMMAND HELPERS
// =====

void sendCommand(const char* command) {
    arduino2Link.println(command);
}

void startRepeatingCommand(const char* command, unsigned int times) {
    repeatCommandText = command;
    repeatCommandRemaining = times;
    repeatCommandActive = true;
    lastRepeatCommandTime = 0;
}

void updateRepeatingCommand() {
    if (!repeatCommandActive) {
        return;
    }

    if (repeatCommandRemaining == 0) {
        repeatCommandActive = false;
    }
}

```



```

    return;
}

unsigned long now = millis();

if (lastRepeatCommandTime == 0 || now - lastRepeatCommandTime >= COMMAND_REPEAT_MS) {
    sendCommand(repeatCommandText);
    lastRepeatCommandTime = now;
    repeatCommandRemaining--;
}
}

void sendTeamToArduino2(TeamColor team) {
    if (team == BLUE_TEAM) {
        sendCommand("TEAM:BLUE");
    } else {
        sendCommand("TEAM:RED");
    }

    lastSentTeam = team;
}

void sendTargetToArduino2(DetectionState state) {
    if (state == DETECT_SELF) {
        sendCommand("TARGET:SELF");
    } else if (state == DETECT_OPPONENT) {
        sendCommand("TARGET:OPPONENT");
    } else {
        sendCommand("TARGET:NONE");
    }

    lastSentDetection = state;
}

void sendCurrentStatusToArduino2() {
    sendTeamToArduino2(currentTeam);
    sendTargetToArduino2(currentDetection);
}

void updateStatusMessages() {
    unsigned long now = millis();

    bool teamChanged = currentTeam != lastSentTeam;
    bool detectionChanged = currentDetection != lastSentDetection;
    bool periodicSend = now - lastStatusSend >= STATUS_SEND_MS;

    if (teamChanged) {
        sendTeamToArduino2(currentTeam);
    }

    if (detectionChanged) {
        sendTargetToArduino2(currentDetection);
    }

    if (periodicSend) {
        sendCurrentStatusToArduino2();
        lastStatusSend = now;
    }
}

// =====
// READ REPLIES FROM ARDUINO 2
// =====
//
// Replies are ignored. This prevents the SoftwareSerial buffer
// from filling up, but does not print anything.
//

void handleArduino2Reply() {
    while (arduino2Link.available()) {
        arduino2Link.read();
    }
}

```

```

}

// =====
// START / STOP SYSTEM
// =====

void startSystem() {
    if (systemIsOn) {
        return;
    }

    systemIsOn = true;
    gameOver = false;

    gameStartTime = millis();
    lastMeasureTime = millis();
    lastLcdUpdate = 0;
    lastStatusSend = 0;

    currentTeam = getTeam();
    currentDetection = NO_SIGNAL;
    lastSentTeam = currentTeam;
    lastSentDetection = currentDetection;

    resetPulseCounters();

    lcdSystemOn();
    setLCDColor(currentTeam);

    printLine(0, "Team: ---");
    printLine(1, "Time: 02:30");

    sendTeamToArduino2(currentTeam);
    sendTargetToArduino2(currentDetection);

    startRepeatingCommand("START", 3);
}

void stopSystem() {
    systemIsOn = false;
    gameOver = false;

    currentDetection = NO_SIGNAL;
    resetPulseCounters();

    lcdSystemOff();

    pinMode(LCD_BLUE_CTRL_PIN, INPUT);

    startRepeatingCommand("STOP", 10);
}

void endGameByTimer() {
    systemIsOn = false;
    gameOver = true;

    currentDetection = NO_SIGNAL;
    resetPulseCounters();

    startRepeatingCommand("STOP", 10);

    printLine(0, "GAME OVER");
    printLine(1, "Time: 00:00");
}

// =====
// DETECTION UPDATE
// =====

void updateDetection(TeamColor team) {
    unsigned long now = millis();

```

```

    if (now - lastMeasureTime >= MEASURE_WINDOW_MS) {
        float freq = getActiveSensorFrequency(team);
        currentDetection = classifyFrequency(freq, team);
        lastMeasureTime = now;
    }
}

// =====
// LCD UPDATE
// =====

void updateRunningLCD(unsigned long elapsed) {
    unsigned long now = millis();

    if (now - lastLcdUpdate >= LCD_UPDATE_MS) {
        printLine(0, detectionText(currentDetection));
        printLine(1, formatTimeRemaining(elapsed));
        lastLcdUpdate = now;
    }
}

void updateGameOverLCD() {
    unsigned long now = millis();

    if (now - lastLcdUpdate >= LCD_UPDATE_MS) {
        setLCDColor(getTeam());
        printLine(0, "GAME OVER");
        printLine(1, "Time: 00:00");
        lastLcdUpdate = now;
    }
}

// =====
// SETUP
// =====

void setup() {
    arduino2Link.begin(9600);

    pinMode(SENSOR1_PIN, INPUT);
    pinMode(SENSOR2_PIN, INPUT);

    pinMode(START_SWITCH_PIN, INPUT_PULLUP);
    pinMode(Team_SWITCH_PIN, INPUT_PULLUP);

    pinMode(LCD_BLUE_CTRL_PIN, INPUT);

    lcd.init();
    lcd.noBacklight();
    lcd.clear();

    attachInterrupt(digitalPinToInterrupt(SENSOR1_PIN), countPulse1, RISING);
    attachInterrupt(digitalPinToInterrupt(SENSOR2_PIN), countPulse2, RISING);

    lastStartSwitchState = startSwitchOn();

    currentTeam = getTeam();
    lastSentTeam = currentTeam;
    currentDetection = NO_SIGNAL;
    lastSentDetection = currentDetection;

    if (lastStartSwitchState) {
        startSystem();
    } else {
        lcdSystemOff();
        startRepeatingCommand("STOP", 10);
    }
}

// =====
// LOOP
// =====

```

```

void loop() {
    handleArduino2Reply();
    updateRepeatingCommand();

    bool currentStartSwitchState = startSwitchOn();

    // OFF -> ON
    if (currentStartSwitchState && !lastStartSwitchState) {
        startSystem();
    }

    // ON -> OFF
    if (!currentStartSwitchState && lastStartSwitchState) {
        stopSystem();
    }

    lastStartSwitchState = currentStartSwitchState;

    // If game ended by timer, keep LCD showing GAME OVER.
    // User must turn toggle OFF, then ON again to restart.
    if (gameOver) {
        updateGameOverLCD();

        if (!currentStartSwitchState) {
            stopSystem();
        }

        return;
    }

    if (!systemIsOn) {
        return;
    }

    currentTeam = getTeam();
    setLCDColor(currentTeam);

    updateDetection(currentTeam);
    updateStatusMessages();

    unsigned long elapsed = millis() - gameStartTime;

    if (elapsed >= GAME_TIME_MS) {
        endGameByTimer();
        return;
    }

    updateRunningLCD(elapsed);
}

```

Arduino_2.ino

```

/*
 * Arduino 2 – Line Follower + Launcher (DUAL PCA9685)
 * GAME LOGIC VERSION
 *
 * Receives commands from Arduino 1 over SoftwareSerial:
 * D10 (RX) ← Arduino 1 D11 (TX)
 * Common GND
 *
 * Commands from Arduino 1:
 * START          – begin run with 10s startup grace period
 * STOP           – kill everything immediately, even mid-launch
 * TEAM:RED       – team is RED → flap 165°, ESC direction A
 * TEAM:BLUE      – team is BLUE → flap 15°, ESC direction B
 * TARGET:SELF    – IR signal from our team's bucket → SKIP (don't score own goal)

```

```

*   TARGET:OPPONENT   - IR signal from opponent's bucket → FIRE (score!)
*   TARGET:NONE       - no IR signal detected → skip
*
* GAME FLOW:
* 1. START → 10s grace (ESC arming, robot stays still)
* 2. Home base bypass: 0.75s straight forward (junction detection disabled)
*   - this skips the junction line at the home base
* 3. Forward line following begins
* 4. Junction detected → stop → wait 5s, collecting TARGET classifications
*   - Count OPPONENT vs SELF votes (ignoring NONE)
*   - If OPPONENT votes > SELF votes AND OPPONENT votes >= 3 → fire
*   - Otherwise → skip junction, resume line following
* 5. After 4 junctions → STOP PERMANENTLY (run ends)
* 6. Stays stopped until next START or STOP command
*
* Hardware:
* PCA9685 #1 @ 0x40 - running at 1000Hz for L298N drive motors
* PCA9685 #2 @ 0x41 - running at 50Hz for ESC + flap servo
* SparkFun Line Follower Array @ 0x3E (front-facing only)
*
* PCA #1 (0x40, drive @ 1000Hz):
*   ch0=IN1, ch1=IN2, ch2=IN3, ch3=IN4, ch4=ENB, ch5=ENA
* PCA #2 (0x41, shooter @ 50Hz):
*   ch0=ESC direction (yellow)
*   ch6=ESC throttle (white)
*   ch7=Flap servo
*/

#include <Wire.h>
#include <sensorbar.h>
#include <Adafruit_PWMServoDriver.h>
#include <SoftwareSerial.h>

// =====
// TOP-LEVEL CONFIGURATION
// =====

// — Serial link —————
const byte ARDUINO1_RX_PIN = 10;
const byte ARDUINO1_TX_PIN = 11;

// — Direction flags —————
#define FLIP_MOTORS true
#define FLIP_SENSOR true

// — Reverse line following strategy —————
// true = Plan A: reverse line following with reversed PID (recommended)
// false = Plan B: just drive straight backwards, no line following
#define USE_PID_REVERSE true

// — ESC direction servo angles (yellow wire) —
const int MOTOR_DIR_RED = 180; // direction for RED team
const int MOTOR_DIR_BLUE = 0; // direction for BLUE team

// Reference angle used during ESC arming.
// Empirically 90 works on this ESC.
const int ESC_ARM_REF_ANGLE = 90;

// — Flap angles per team —————
const int FLAP_CENTER = 90; // resting / closed
const int FLAP_OPEN_RED = 165; // red team release angle
const int FLAP_OPEN_BLUE = 15; // blue team release angle

// — Forward PID tuning —————
float KP = 0.35;
float KI = 0.00;
float KD = 0.4;

// — Reverse PID tuning (smaller gains for stability) —
float KP_REV = 0.25;
float KI_REV = 0.00;
float KD_REV = 0.20;

```

```

// — Drive speeds (PCA9685 0-4095) —————
int BASE_SPEED      = 1100;
int CURVE_SPEED     = 1500;
int PIVOT_SPEED     = 3000;
int SEARCH_SPEED    = 1200;
int MAX_SPEED       = 3000;
int MIN_MOTOR_SPEED = 800;

// Reverse-mode speeds (slower for stability)
int BASE_SPEED_REV  = 600;
int CURVE_SPEED_REV = 800;
int PIVOT_SPEED_REV = 1500;

int START_BOOST_SPEED = 2300;
unsigned long START_BOOST_MS = 180;

float CENTER_DEADBAND = 0.05;
float CURVE_THRESHOLD = 1.0;
float PIVOT_THRESHOLD = 1.8;

// — Junction settings —————
int JUNCTION_SENSOR_COUNT      = 5;
unsigned long JUNCTION_DRIVE_MS = 200;
int JUNCTION_FORWARD_SPEED     = 1200;
unsigned long JUNCTION_COOLDOWN_MS = 800;
const int TOTAL_JUNCTIONS      = 4; // stop permanently after this many junctions

// — Target detection wait at junction —————
const unsigned long TARGET_WAIT_MS = 5000; // 5s window to collect votes
const int MIN_OPPONENT_VOTES = 3;          // need at least this many OPPONENT votes
                                           // AND OPPONENT > SELF to fire

// — Junction skip (no fire) settings —————
const unsigned long SKIP_PAUSE_MS = 200;          // brief pause before resuming
const unsigned long SKIP_BOOST_MS = 250;          // boost forward to clear junction
const int SKIP_BOOST_SPEED = 1500;

// — Shooter timing —————
const float TARGET_THROTTLE = 0.27;

// 10s startup grace period after START.
const unsigned long STARTUP_GRACE_MS = 3000;
const unsigned long ESC_ARM_MS       = 3000;

// After grace ends, drive forward at HOME_BASE_BYPASS_SPEED for
// HOME_BASE_BYPASS_MS to physically clear the home base line.
// Then keep junction/sensor detection DISABLED for SENSOR_IGNORE_MS
// (measured from the start of bypass) – this prevents the home base
// line from being counted as a junction even after the robot resumes
// normal line following.
const unsigned long HOME_BASE_BYPASS_MS = 150; // motion duration (boost forward)
const int HOME_BASE_BYPASS_SPEED = 1800;      // motion speed
const unsigned long SENSOR_IGNORE_MS = 800;   // junction detection stays off this long
                                           // (must be >= HOME_BASE_BYPASS_MS)

// Per-launch timing
const unsigned long DIR_SETTLE_MS = 300;
const unsigned long SPINUP_WAIT_MS = 1500;
const unsigned long FLAP_HOLD_MS = 600;
const unsigned long FLAP_RETURN_MS = 600;
const unsigned long POST_FLAP_MS = 1500;
const unsigned long POST_LAUNCH_MS = 2000;

// — PCA9685 #1 channel mapping (drive @ 1000Hz, addr 0x40) —
#define CH_IN1 0
#define CH_IN2 1
#define CH_ENA 5
#define CH_IN3 2
#define CH_IN4 3
#define CH_ENB 4

```

```

// — PCA9685 #2 channel mapping (shooter @ 50Hz, addr 0x41) —
const int ESC_DIR_CHANNEL = 0;
const int ESC_THR_CHANNEL = 6;
const int FLAP_CHANNEL    = 7;

// =====
// GLOBALS
// =====
SoftwareSerial arduino1Link(ARDUINO1_RX_PIN, ARDUINO1_TX_PIN);

SensorBar bar(0x3E);
Adafruit_PWMServoDriver pwmDrive(0x40);
Adafruit_PWMServoDriver pwmShoot(0x41);

enum TeamColor { TEAM_RED, TEAM_BLUE };
TeamColor currentTeam = TEAM_RED;

enum TargetState { TARGET_NONE, TARGET_SELF, TARGET_OPPONENT };
TargetState currentTarget = TARGET_NONE;

bool running = false;
unsigned long runStartTime = 0;
bool inStartupGrace = false;

// True while we're in the home base bypass phase (driving forward with
// junction detection disabled) immediately after grace ends.
bool inHomeBaseBypass = false;
unsigned long homeBaseBypassStart = 0;

// Forward (true) or reverse (false) line-following direction
bool forwardDirection = true;

bool escArmed = false;

enum GracePhase {
    GRACE_INIT,
    GRACE_ARMING,
    GRACE_IDLE
};
GracePhase gracePhase = GRACE_INIT;
unsigned long gracePhaseStart = 0;

// PID state (shared across forward and reverse – reset on direction flip)
float integral = 0.0;
float lastError = 0.0;
int lastKnownDir = 0;
int lastLeftSpeed = 0;
int lastRightSpeed = 0;
bool wasInTurn = false;

// Junction state
bool junctionDetected = false;
unsigned long junctionStartTime = 0;
unsigned long lastJunctionTime = 0;
int junctionCount = 0; // resets every JUNCTIONS_PER_DIRECTION

// Vote counters for the current junction wait window.
// Cleared at the start of each junction wait. Incremented every time
// a TARGET:* message arrives. NONE is counted but ignored in the decision.
int votesOpponent = 0;
int votesSelf = 0;
int votesNone = 0;

// =====
// STATE MACHINES
// =====

// What to do after stopping at a junction
enum JunctionPhase {
    JUNC_IDLE,
    JUNC_APPROACH, // brief forward drive after detection (to align over junction)
    JUNC_WAIT_FOR_TARGET, // 5s window: collecting TARGET votes

```

```

    JUNC_SKIP_PAUSE,          // brief pause when not firing
    JUNC_SKIP_BOOST           // boost through to leave the junction
};
JunctionPhase junctionPhase = JUNC_IDLE;
unsigned long junctionPhaseStart = 0;

// Launch state machine (entered when SELF is confirmed at junction)
enum LaunchPhase {
    LAUNCH_IDLE,
    LAUNCH_SET_DIRECTION,
    LAUNCH_DIR_SETTLE,
    LAUNCH_SPINUP,
    LAUNCH_FLAP_OPEN,
    LAUNCH_FLAP_RETURN,
    LAUNCH_POST_FLAP,
    LAUNCH_ESC_OFF,
    LAUNCH_POST_LAUNCH,
    LAUNCH_RESUME_BOOST
};
LaunchPhase launchPhase = LAUNCH_IDLE;
unsigned long launchPhaseStart = 0;

// =====
//  PCA9685 HELPERS
// =====
int usToCounts50Hz(int us) {
    return (int)((us * 4096L) / 20000L);
}

void setShooterPulseUs(int channel, int us) {
    pwmShoot.setPWM(channel, 0, usToCounts50Hz(us));
}

// — Cached last-written values for shooter board (50Hz) —
// PCA9685 holds whatever value was last written until you change it.
// Continuously re-writing the same value every loop iteration is
// wasted I2C traffic AND can cause servo jitter on cheap servos
// because each I2C transaction has small timing variation.
// We cache the last value sent for each channel and only re-write
// when it actually changes. -1 means "never written yet" so the
// first call always goes through.
int lastEscThrottleUs = -1;
int lastEscDirAngleDeg = -1;
int lastFlapAngleDeg = -1;

void setEsc(float throttle) {
    if (throttle < 0.0) throttle = 0.0;
    if (throttle > 1.0) throttle = 1.0;
    int us = 1000 + (int)(throttle * 1000.0);
    if (us == lastEscThrottleUs) return; // no-op if unchanged
    setShooterPulseUs(ESC_THR_CHANNEL, us);
    lastEscThrottleUs = us;
}

void setServoAngle(int channel, int degrees) {
    if (degrees < 0) degrees = 0;
    if (degrees > 180) degrees = 180;
    int us = 1000 + (int)((degrees / 180.0) * 1000.0);
    setShooterPulseUs(channel, us);
}

void setFlap(int degrees) {
    if (degrees < 0) degrees = 0;
    if (degrees > 180) degrees = 180;
    if (degrees == lastFlapAngleDeg) return; // no-op if unchanged
    setServoAngle(FLAP_CHANNEL, degrees);
    lastFlapAngleDeg = degrees;
}

void setMotorDirectionForTeam() {
    int angle = (currentTeam == TEAM_RED) ? MOTOR_DIR_RED : MOTOR_DIR_BLUE;
    if (angle == lastEscDirAngleDeg) return; // no-op if unchanged

```



```

    setServoAngle(ESC_DIR_CHANNEL, angle);
    lastEscDirAngleDeg = angle;
}

void setEscDirectionToArmRef() {
    if (ESC_ARM_REF_ANGLE == lastEscDirAngleDeg) return;    // no-op if unchanged
    setServoAngle(ESC_DIR_CHANNEL, ESC_ARM_REF_ANGLE);
    lastEscDirAngleDeg = ESC_ARM_REF_ANGLE;
}

int getFlapOpenAngleForTeam() {
    return (currentTeam == TEAM_RED) ? FLAP_OPEN_RED : FLAP_OPEN_BLUE;
}

void forceShooterChannelsOff() {
    pwmShoot.setPWM(ESC_THR_CHANNEL, 0, 4096);
    pwmShoot.setPWM(FLAP_CHANNEL, 0, 4096);
    // After explicitly cutting signals, invalidate caches so the next
    // setEsc()/setFlap() call definitely writes through.
    lastEscThrottleUs = -1;
    lastFlapAngleDeg = -1;
}

// =====
// DRIVE MOTOR FUNCTIONS (PCA #1 @ 1000Hz)
// =====
void pinHigh(uint8_t ch) { pwmDrive.setPWM(ch, 4096, 0); }
void pinLow (uint8_t ch) { pwmDrive.setPWM(ch, 0, 4096); }

void setSpeed(uint8_t ch, int speed) {
    speed = constrain(speed, 0, 4095);
    pwmDrive.setPWM(ch, 0, speed);
}

void stopMotors() {
    setSpeed(CH_ENA, 0);
    setSpeed(CH_ENB, 0);
    pinLow(CH_IN1);
    pinLow(CH_IN2);
    pinLow(CH_IN3);
    pinLow(CH_IN4);
}

void setMotors(int leftSpeed, int rightSpeed) {
    if (FLIP_MOTORS) {
        leftSpeed = -leftSpeed;
        rightSpeed = -rightSpeed;
    }

    leftSpeed = constrain(leftSpeed, -MAX_SPEED, MAX_SPEED);
    rightSpeed = constrain(rightSpeed, -MAX_SPEED, MAX_SPEED);

    if (leftSpeed > 0 && leftSpeed < MIN_MOTOR_SPEED) leftSpeed = MIN_MOTOR_SPEED;
    if (leftSpeed < 0 && leftSpeed > -MIN_MOTOR_SPEED) leftSpeed = -MIN_MOTOR_SPEED;
    if (rightSpeed > 0 && rightSpeed < MIN_MOTOR_SPEED) rightSpeed = MIN_MOTOR_SPEED;
    if (rightSpeed < 0 && rightSpeed > -MIN_MOTOR_SPEED) rightSpeed = -MIN_MOTOR_SPEED;

    if (leftSpeed >= 0) {
        pinHigh(CH_IN1);
        pinLow(CH_IN2);
    } else {
        pinLow(CH_IN1);
        pinHigh(CH_IN2);
    }
    setSpeed(CH_ENA, abs(leftSpeed));

    if (rightSpeed >= 0) {
        pinLow(CH_IN3);
        pinHigh(CH_IN4);
    } else {
        pinHigh(CH_IN3);
        pinLow(CH_IN4);
    }
}

```

```

    }
    setSpeed(CH_ENB, abs(rightSpeed));
}

// =====
//  SENSOR FUNCTIONS
// =====
bool sensorOn(uint8_t raw, int n) { return raw & (1 << n); }

void printRawBits(uint8_t raw) {
    for (int i = 7; i >= 0; i--) Serial.print(sensorOn(raw, i) ? "1" : "0");
}

int countActiveSensors(uint8_t raw) {
    int c = 0;
    for (int i = 0; i < 8; i++) if (sensorOn(raw, i)) c++;
    return c;
}

float getLinePosition() {
    uint8_t raw = bar.getRaw();
    if (raw == 0) return 99.0;

    float weightedSum = 0.0;
    float totalWeight = 0.0;
    for (int i = 0; i < 8; i++) {
        if (sensorOn(raw, i)) {
            float pos = FLIP_SENSOR ? ((float)i - 3.5) : (3.5 - (float)i);
            weightedSum += pos;
            totalWeight += 1.0;
        }
    }
    return weightedSum / totalWeight;
}

// =====
//  STATE RESET
// =====
void resetStateForStart() {
    integral = 0.0;
    lastError = 0.0;
    lastKnownDir = 0;
    lastLeftSpeed = 0;
    lastRightSpeed = 0;
    wasInTurn = false;
    junctionDetected = false;
    junctionStartTime = 0;
    lastJunctionTime = millis();
    junctionCount = 0;
    launchPhase = LAUNCH_IDLE;
    junctionPhase = JUNC_IDLE;
    forwardDirection = true;
    currentTarget = TARGET_NONE;
    votesOpponent = 0;
    votesSelf = 0;
    votesNone = 0;
    escArmed = false;
    inHomeBaseBypass = false;
    // Invalidate PWM caches so the first write of this run definitely
    // goes through (in case something was previously written that
    // happens to match the new value we want to send).
    lastEscThrottleUs = -1;
    lastEscDirAngleDeg = -1;
    lastFlapAngleDeg = -1;
}

void killAll() {
    // Order matters: stop the mechanism that's most dangerous first.
    // ESC + drive motors first, then everything else.
    setEsc(0.0);           // throttle to zero immediately
    stopMotors();          // drive motors off
    setFlap(FLAP_CENTER);  // flap to neutral
}

```

```

    forceShooterChannelsOff();    // explicitly cut shooter PWM signals

    running = false;
    launchPhase = LAUNCH_IDLE;
    junctionPhase = JUNC_IDLE;
    junctionDetected = false;
    inStartupGrace = false;
    inHomeBaseBypass = false;
    gracePhase = GRACE_INIT;
    escArmed = false;

    // Clear vote counters so a stale state doesn't carry over
    votesOpponent = 0;
    votesSelf = 0;
    votesNone = 0;

    // Invalidate PWM caches – the next setEsc/setFlap/setServoAngle
    // call should always write through, since forceShooterChannelsOff()
    // bypassed the cache and put hardware in an unknown state.
    lastEscThrottleUs = -1;
    lastEscDirAngleDeg = -1;
    lastFlapAngleDeg = -1;

    Serial.println(F("[killAll] all systems stopped"));
}

// =====
//  COMMAND HANDLERS
//  =====
void handleStart() {
    if (running) return;
    Serial.println(F("=== START – entering 10s startup grace ==="));
    resetStateForStart();
    running = true;

    stopMotors();
    setEsc(0.0);
    setFlap(FLAP_CENTER);

    runStartTime = millis();
    inStartupGrace = true;

    gracePhase = GRACE_INIT;
    gracePhaseStart = millis();
}

void handleStop() {
    Serial.println(F("=== STOP ==="));
    killAll();
}

void handleSerial() {
    while (arduino1Link.available()) {
        String cmd = arduino1Link.readStringUntil('\n');
        cmd.trim();
        if (cmd.length() == 0) continue;

        if (cmd == "START") {
            handleStart();
        } else if (cmd == "STOP") {
            handleStop();
        } else if (cmd == "TEAM:RED") {
            bool changed = (currentTeam != TEAM_RED);
            currentTeam = TEAM_RED;
            if (escArmed) {
                setMotorDirectionForTeam();
            }
            if (changed) {
                Serial.print(F("Team: RED | DIR angle = "));
                Serial.print(MOTOR_DIR_RED);
                Serial.println(escArmed ? F(" (applied)") : F(" (deferred until armed)"));
            }
        }
    }
}

```

```

    } else if (cmd == "TEAM:BLUE") {
        bool changed = (currentTeam != TEAM_BLUE);
        currentTeam = TEAM_BLUE;
        if (escArmed) {
            setMotorDirectionForTeam();
        }
        if (changed) {
            Serial.print(F("Team: BLUE | DIR angle = "));
            Serial.print(MOTOR_DIR_BLUE);
            Serial.println(escArmed ? F(" (applied)") : F(" (deferred until armed)"));
        }
    } else if (cmd == "TARGET:SELF") {
        currentTarget = TARGET_SELF;
        if (junctionPhase == JUNC_WAIT_FOR_TARGET) {
            votesSelf++;
        }
    } else if (cmd == "TARGET:OPPONENT") {
        currentTarget = TARGET_OPPONENT;
        if (junctionPhase == JUNC_WAIT_FOR_TARGET) {
            votesOpponent++;
        }
    } else if (cmd == "TARGET:NONE") {
        currentTarget = TARGET_NONE;
        if (junctionPhase == JUNC_WAIT_FOR_TARGET) {
            votesNone++;
        }
    }
}
}

// =====
// GRACE PERIOD STATE MACHINE
// =====
void updateGrace(unsigned long now) {
    unsigned long elapsed = now - gracePhaseStart;

    switch (gracePhase) {

        case GRACE_INIT:
            setEscDirectionToArmRef();
            setEsc(0.0);
            gracePhase = GRACE_ARMING;
            gracePhaseStart = now;
            Serial.print(F("[GRACE] arming ESC at reference angle "));
            Serial.println(ESC_ARM_REF_ANGLE);
            break;

        case GRACE_ARMING:
            setEscDirectionToArmRef();
            setEsc(0.0);
            if (elapsed >= ESC_ARM_MS) {
                escArmed = true;
                setMotorDirectionForTeam();
                gracePhase = GRACE_IDLE;
                gracePhaseStart = now;
                Serial.print(F("[GRACE] ESC armed - direction set for team "));
                Serial.print(currentTeam == TEAM_RED ? F("RED ") : F("BLUE "));
                Serial.print(currentTeam == TEAM_RED ? MOTOR_DIR_RED : MOTOR_DIR_BLUE);
                Serial.println(F(""));
            }
            break;

        case GRACE_IDLE:
            break;
    }
}

// =====
// JUNCTION HANDLING STATE MACHINE
// =====
void enterJunctionWait() {
    Serial.print(F("Junction #"));

```

```

Serial.print(junctionCount);
Serial.println(F(" - driving forward briefly to align over junction..."));
// Don't stop yet - drive forward a bit to position the launcher
// properly over the bucket. JUNC_APPROACH will hold this for
// JUNCTION_DRIVE_MS, then transition to vote collection.
setMotors(JUNCTION_FORWARD_SPEED, JUNCTION_FORWARD_SPEED);
// Clear vote counters - only votes during the wait phase count.
votesOpponent = 0;
votesSelf = 0;
votesNone = 0;
junctionPhase = JUNC_APPROACH;
junctionPhaseStart = millis();
}

void updateJunction() {
  // Always check for STOP first - junction wait can be up to 5s.
  handleSerial();
  if (!running) {
    return; // killAll() already shut everything down
  }

  unsigned long now = millis();
  unsigned long elapsed = now - junctionPhaseStart;

  switch (junctionPhase) {

    case JUNC_APPROACH:
      // Drive forward briefly so the launcher ends up properly aligned
      // with the bucket (the IR sensor leads the body, so by the time
      // we detect the junction we're not yet centered on it).
      setMotors(JUNCTION_FORWARD_SPEED, JUNCTION_FORWARD_SPEED);
      if (elapsed >= JUNCTION_DRIVE_MS) {
        Serial.println(F(" -> aligned, collecting TARGET votes for 5s..."));
        stopMotors();
        // Reset vote counters again here in case any TARGET messages
        // arrived during the approach phase - we only want votes from
        // the actual wait window.
        votesOpponent = 0;
        votesSelf = 0;
        votesNone = 0;
        junctionPhase = JUNC_WAIT_FOR_TARGET;
        junctionPhaseStart = now;
      }
      break;

    case JUNC_WAIT_FOR_TARGET:
      stopMotors();
      // Wait the full TARGET_WAIT_MS to collect votes - this gives time
      // to handle:
      // 1. Noisy classifications (e.g. RED@1500Hz briefly mis-read as 750Hz)
      // 2. IR LED/sensor misalignment causing intermittent NONE readings
      // Then decide based on majority vote.
      if (elapsed >= TARGET_WAIT_MS) {
        Serial.print(F(" votes: OPP="));
        Serial.print(votesOpponent);
        Serial.print(F(" SELF="));
        Serial.print(votesSelf);
        Serial.print(F(" NONE="));
        Serial.println(votesNone);

        // Fire only if BOTH:
        // (a) OPPONENT votes > SELF votes (majority among non-NONE)
        // (b) OPPONENT votes >= MIN_OPPONENT_VOTES (enough confidence)
        bool shouldFire = (votesOpponent > votesSelf) &&
          (votesOpponent >= MIN_OPPONENT_VOTES);

        if (shouldFire) {
          Serial.println(F(" -> FIRING (OPPONENT wins majority)"));
          junctionPhase = JUNC_IDLE;
          launchPhase = LAUNCH_SET_DIRECTION;
          launchPhaseStart = now;
        } else if (junctionCount >= TOTAL_JUNCTIONS) {

```

```

        // Final junction, not firing – just stop here, no boost.
        Serial.println(F(" -> SKIPPING + final junction, stopping in place"));
        stopMotors();
        junctionPhase = JUNC_IDLE;
        lastJunctionTime = now;
    } else {
        Serial.println(F(" -> SKIPPING (insufficient OPPONENT votes)"));
        junctionPhase = JUNC_SKIP_PAUSE;
        junctionPhaseStart = now;
    }
}
break;

case JUNC_SKIP_PAUSE:
    stopMotors();
    if (elapsed >= SKIP_PAUSE_MS) {
        junctionPhase = JUNC_SKIP_BOOST;
        junctionPhaseStart = now;
    }
    break;

case JUNC_SKIP_BOOST: {
    // Boost in current travel direction to clear the junction
    int s = forwardDirection ? SKIP_BOOST_SPEED : -SKIP_BOOST_SPEED;
    setMotors(s, s);
    if (elapsed >= SKIP_BOOST_MS) {
        junctionPhase = JUNC_IDLE;
        lastJunctionTime = now;
        // Check if we just hit JUNCTIONS_PER_DIRECTION – handled in main loop
    }
    break;
}

default:
    break;
}
}

// =====
// LAUNCH STATE MACHINE
// =====
void updateLaunch() {
    // Always check for STOP first – launch sequence is long (~7s total)
    // and we want STOP to abort it cleanly.
    handleSerial();
    if (!running) {
        return; // killAll() already shut everything down
    }

    unsigned long now = millis();
    unsigned long elapsed = now - launchPhaseStart;

    switch (launchPhase) {

        case LAUNCH_SET_DIRECTION:
            setEsc(0.0);
            setMotorDirectionForTeam();
            launchPhase = LAUNCH_DIR_SETTLE;
            launchPhaseStart = now;
            break;

        case LAUNCH_DIR_SETTLE:
            if (elapsed >= DIR_SETTLE_MS) {
                setEsc(TARGET_THROTTLE);
                launchPhase = LAUNCH_SPINUP;
                launchPhaseStart = now;
            }
            break;

        case LAUNCH_SPINUP:
            if (elapsed >= SPINUP_WAIT_MS) {
                setFlap(getFlapOpenAngleForTeam());
            }

```

```

        launchPhase = LAUNCH_FLAP_OPEN;
        launchPhaseStart = now;
    }
    break;

case LAUNCH_FLAP_OPEN:
    if (elapsed >= FLAP_HOLD_MS) {
        setFlap(FLAP_CENTER);
        launchPhase = LAUNCH_FLAP_RETURN;
        launchPhaseStart = now;
    }
    break;

case LAUNCH_FLAP_RETURN:
    if (elapsed >= FLAP_RETURN_MS) {
        launchPhase = LAUNCH_POST_FLAP;
        launchPhaseStart = now;
    }
    break;

case LAUNCH_POST_FLAP:
    if (elapsed >= POST_FLAP_MS) {
        setEsc(0.0);
        launchPhase = LAUNCH_ESC_OFF;
        launchPhaseStart = now;
    }
    break;

case LAUNCH_ESC_OFF:
    if (elapsed >= 200) {
        forceShooterChannelsOff();
        launchPhase = LAUNCH_POST_LAUNCH;
        launchPhaseStart = now;
    }
    break;

case LAUNCH_POST_LAUNCH:
    if (elapsed >= POST_LAUNCH_MS) {
        // If this was the final junction, don't boost – just stop here
        // and let the main loop transition to the permanent stop state.
        if (junctionCount >= TOTAL_JUNCTIONS) {
            stopMotors();
            launchPhase = LAUNCH_IDLE;
            lastJunctionTime = now;
            Serial.print(F("Final junction #"));
            Serial.print(junctionCount);
            Serial.println(F(" – launch done, no boost (stopping in place)"));
        } else {
            // Normal junction: boost forward to clear it
            int s = forwardDirection ? START_BOOST_SPEED : -START_BOOST_SPEED;
            setMotors(s, s);
            launchPhase = LAUNCH_RESUME_BOOST;
            launchPhaseStart = now;
            lastJunctionTime = now;
        }
    }
    break;

case LAUNCH_RESUME_BOOST:
    if (elapsed >= START_BOOST_MS) {
        launchPhase = LAUNCH_IDLE;
        Serial.print(F("Resumed after junction #"));
        Serial.println(junctionCount);
    }
    break;

default:
    break;
}
}

// =====

```

```

// LINE FOLLOWING (handles both forward and reverse)
// =====
void doLineFollowing() {
    unsigned long now = millis();
    uint8_t raw = bar.getRaw();
    float position = getLinePosition();
    int activeSensors = countActiveSensors(raw);

    // Are we still inside the post-bypass sensor-ignore window?
    // homeBaseBypassStart is set when bypass begins; junction detection
    // is gated for SENSOR_IGNORE_MS from that point. PID line following
    // still runs – only junction detection is suppressed.
    bool sensorIgnoreActive = (now - homeBaseBypassStart < SENSOR_IGNORE_MS);

    // Junction detection – gated by SENSOR_IGNORE_MS and JUNCTION_COOLDOWN_MS
    if (!sensorIgnoreActive &&
        activeSensors >= JUNCTION_SENSOR_COUNT &&
        (now - lastJunctionTime > JUNCTION_COOLDOWN_MS)) {

        junctionCount++;
        lastJunctionTime = now;

        Serial.print(F("JUNCTION #"));
        Serial.print(junctionCount);
        Serial.print(F("/"));
        Serial.print(TOTAL_JUNCTIONS);
        Serial.print(F(" sensors="));
        Serial.print(activeSensors);
        Serial.print(F(" raw="));
        printRawBits(raw);
        Serial.print(F(" mode="));
        Serial.println(forwardDirection ? F("FWD") : F("REV"));

        integral = 0.0;
        lastError = 0.0;
        wasInTurn = false;

        // Go into target wait – vote and decide whether to fire.
        // After this junction is fully handled (fire or skip), if we've
        // hit TOTAL_JUNCTIONS we'll stop permanently in the main loop.
        enterJunctionWait();
        return;
    }

    // — Plan B: simple straight reverse, no PID —
    if (!forwardDirection && !USE_PID_REVERSE) {
        setMotors(-BASE_SPEED_REV, -BASE_SPEED_REV);
        return;
    }

    // — Pick PID gains and speeds based on direction —
    float kp = forwardDirection ? KP : KP_REV;
    float ki = forwardDirection ? KI : KI_REV;
    float kd = forwardDirection ? KD : KD_REV;
    int baseSpd = forwardDirection ? BASE_SPEED : BASE_SPEED_REV;
    int curveSpd = forwardDirection ? CURVE_SPEED : CURVE_SPEED_REV;
    int pivotSpd = forwardDirection ? PIVOT_SPEED : PIVOT_SPEED_REV;

    // CASE 1: line lost
    if (position == 99.0) {
        integral = 0.0;
        lastError = 0.0;

        if (wasInTurn) {
            // Replay last turn motion (already in correct direction)
            setMotors(lastLeftSpeed, lastRightSpeed);
        } else if (lastKnownDir < 0) {
            int l = -SEARCH_SPEED;
            int r = SEARCH_SPEED;
            if (!forwardDirection) { l = -l; r = -r; }
            setMotors(l, r);
            lastLeftSpeed = l;
        }
    }
}

```



```

        lastRightSpeed = r;
        wasInTurn = true;
    } else if (lastKnownDir > 0) {
        int l = SEARCH_SPEED;
        int r = -SEARCH_SPEED;
        if (!forwardDirection) { l = -l; r = -r; }
        setMotors(l, r);
        lastLeftSpeed = l;
        lastRightSpeed = r;
        wasInTurn = true;
    } else {
        int s = forwardDirection ? MIN_MOTOR_SPEED : -MIN_MOTOR_SPEED;
        setMotors(s, s);
    }
    return;
}

// CASE 2: 90 degree turn (sharp)
if (abs(position) >= PIVOT_THRESHOLD) {
    integral = 0.0;
    lastError = 0.0;

    int l, r;
    if (position < 0) {
        lastKnownDir = -1;
        l = -pivotSpd;
        r = pivotSpd;
    } else {
        lastKnownDir = 1;
        l = pivotSpd;
        r = -pivotSpd;
    }
    // For pivot, both directions need to rotate the same way relative to
    // the line. Pivots are symmetric (one wheel forward, other backward),
    // so no sign flip needed for reverse mode – we still want to align
    // with the line as seen by the front sensor.
    setMotors(l, r);
    lastLeftSpeed = l;
    lastRightSpeed = r;
    wasInTurn = true;
    return;
}

// CASE 3: normal PID line following
float error = position;
if (abs(error) < CENTER_DEADBAND) error = 0.0;

if (error < -0.15) lastKnownDir = -1;
else if (error > 0.15) lastKnownDir = 1;

// For reverse mode, flip the error sign so PID corrects in the right
// sense (rear of car moves toward the line, instead of front).
float pidError = forwardDirection ? error : -error;

float P = kp * pidError;
if ((pidError > 0 && lastError < 0) || (pidError < 0 && lastError > 0)) {
    integral = 0.0;
}
integral += pidError;
integral = constrain(integral, -25.0, 25.0);

float I = ki * integral;
float D = kd * (pidError - lastError);
lastError = pidError;

float correction = P + I + D;

int currentBaseSpeed;
if (abs(position) > CURVE_THRESHOLD) {
    currentBaseSpeed = curveSpd;
    wasInTurn = true;
} else {

```

```

    currentBaseSpeed = baseSpd;
    wasInTurn = false;
}

int leftSpeed = currentBaseSpeed + (int)(correction * (float)currentBaseSpeed);
int rightSpeed = currentBaseSpeed - (int)(correction * (float)currentBaseSpeed);

// Reverse mode: negate both motor outputs (drive backwards)
if (!forwardDirection) {
    leftSpeed = -leftSpeed;
    rightSpeed = -rightSpeed;
}

leftSpeed = constrain(leftSpeed, -MAX_SPEED, MAX_SPEED);
rightSpeed = constrain(rightSpeed, -MAX_SPEED, MAX_SPEED);

lastLeftSpeed = leftSpeed;
lastRightSpeed = rightSpeed;

setMotors(leftSpeed, rightSpeed);
}

// =====
//  SETUP
// =====
void setup() {
    Serial.begin(9600);
    arduino1Link.begin(9600);
    arduino1Link.setTimeout(50);

    Wire.begin();

    pwmDrive.begin();
    pwmDrive.setOscillatorFrequency(25000000);
    pwmDrive.setPWMFreq(1000);

    pwmShoot.begin();
    pwmShoot.setOscillatorFrequency(25000000);
    pwmShoot.setPWMFreq(50);

    delay(100);

    stopMotors();
    setEsc(0.0);
    setFlap(FLAP_CENTER);

    setEscDirectionToArmRef();
    escArmed = false;

    if (bar.begin() == false) {
        Serial.println(F("ERROR: line follower array not found.));
        while (1) {
            stopMotors();
            handleSerial();
            delay(200);
        }
    }

    Serial.println(F("=== READY - waiting for START ==="));
}

// =====
//  MAIN LOOP
// =====
void loop() {
    // Always check serial first - STOP must be honoured immediately
    handleSerial();

    if (!running) {
        return;
    }
}

```

```

    unsigned long now = millis();

    // — Startup grace period —
    if (inStartupGrace) {
        if (now - runStartTime >= STARTUP_GRACE_MS) {
            Serial.println(F("=== Grace done – entering home base bypass ==="));
            inStartupGrace = false;
            inHomeBaseBypass = true;
            homeBaseBypassStart = now;
            // Start driving forward immediately. We'll keep driving and
            // ignoring the line/junction sensors until HOME_BASE_BYPASS_MS
            // elapses, so the line at the home base doesn't get counted
            // as a junction.
            setMotors(HOME_BASE_BYPASS_SPEED, HOME_BASE_BYPASS_SPEED);
        } else {
            stopMotors();
            updateGrace(now);
            return;
        }
    }

    // — Home base bypass: drive forward at boost speed, ignore everything —
    if (inHomeBaseBypass) {
        if (now - homeBaseBypassStart >= HOME_BASE_BYPASS_MS) {
            Serial.println(F("=== Home base bypass motion done – line following begins (junction
detection still off) ==="));
            inHomeBaseBypass = false;
            // Reset PID state for clean line following start.
            integral = 0.0;
            lastError = 0.0;
            wasInTurn = false;
            // NOTE: we don't return here – fall through to doLineFollowing()
            // so the robot starts following the line immediately. Junction
            // detection is gated separately by SENSOR_IGNORE_MS (see
            // doLineFollowing) so the home base line isn't counted as a junction.
        } else {
            // Keep driving forward at boost speed – DO NOT read sensors at all
            setMotors(HOME_BASE_BYPASS_SPEED, HOME_BASE_BYPASS_SPEED);
            return;
        }
    }

    // — Launch sequence in progress? —
    if (launchPhase != LAUNCH_IDLE) {
        updateLaunch();
        return;
    }

    // — Junction handling (target wait / skip) in progress? —
    if (junctionPhase != JUNC_IDLE) {
        updateJunction();
        return;
    }

    // — Done with TOTAL_JUNCTIONS junctions? Stop permanently. —
    // We check this here (after junction/launch phases have returned to IDLE)
    // so the last junction's launch or skip sequence completes cleanly.
    if (junctionCount >= TOTAL_JUNCTIONS) {
        Serial.print(F("=== "));
        Serial.print(TOTAL_JUNCTIONS);
        Serial.println(F(" junctions reached – STOPPING ==="));
        killAll();
        return;
    }

    // — Normal line following (forward or reverse) —
    doLineFollowing();
}

```

Circuit Schematic (KiCAD)

